

Received 18 January 2026, accepted 1 February 2026, date of publication 10 February 2026, date of current version 13 February 2026.

Digital Object Identifier 10.1109/ACCESS.2026.3663379

RESEARCH ARTICLE

TICNN—A Hybrid Light-Weight CNN for Large Imbalanced Multi-Class Datasets on Intrusion Detection System in IoT

AVINASH AWASTHI¹, (Member, IEEE), ANIL KUMAR PRAJAPATI², (Member, IEEE),
PRITAM VEDIYA¹, AND RAMESH BABU BATTULA¹, (Member, IEEE)

¹Department of Computer Science and Engineering, Malaviya National Institute of Technology (MNIT), Jaipur, Rajasthan 302017, India

²Department of Data Science and Engineering, Manipal University Jaipur, Rajasthan 303007, India

Corresponding author: Anil Kumar Prajapati (anilkumar.prajapati@jaipur.manipal.edu)

This work was supported by the Manipal University Jaipur, Jaipur, Rajasthan, India.

ABSTRACT The swift growth in ubiquitous connectivity, such as 5G and 6G, cloud computing, which enables data-driven automation and increases efficiency across industries, gave rise to the Internet of Things. This proliferation significantly expands the network attack surface, making IoT-based networks increasingly vulnerable to a multitude of attacks. To mitigate the attacks, Intrusion detection systems are developed. While developing the IDS, a primary challenge is the data collection of attack patterns and their training. The data collected is often massive, heterogeneous, and, critically, imbalanced. Training conventional models on such imbalanced data typically leads to poor performance, particularly in overlooking minority attack classes during multi-class classification. Our proposed system directly addresses this challenge of data imbalance. We introduce a hybrid technique, the Transformed Image Generative Adversarial Network (TIGAN), that generates high-fidelity attack samples and is then used to augment underrepresented attack classes, complemented by downsampling of overrepresented ones. The balanced data allows our lightweight deep learning model, Transformed Image Convolutional Neural Network (TICNN), to achieve robust attack detection. The experimental TICNN-based IDS delivers a faster detection time and detection accuracy of 99.56% at the macro level, which showcases its strong capability to identify diverse attacks, sporadic minority classes, in challenging IoT settings.

INDEX TERMS IoT threat vector, generative adversarial networks (GANs), multi-class classification, deep transfer learning, convolutional neural network, IDS.

I. INTRODUCTION

The Internet of Things (IoT) is rapidly growing due to enhanced connectivity, automation, and data analysis in sectors such as manufacturing (Industrial IoT), healthcare, smart cities, retail, agriculture, transportation and logistics, energy and utilities, and smart homes. The developments in next-generation systems have improved connectivity, helping IoT devices communicate seamlessly, enabling real-time data exchange, and improving operational performance. However, IoT devices often lack adequate security, as functionality may

be prioritized over protective measures. The sheer number of connected devices within an IoT ecosystem creates a vast attack surface, making them vulnerable to cyberattacks, unauthorised access, data breaches, etc. Thus, Real-time detection of attacks is crucial for safeguarding IoT systems. Machine learning-based intrusion detection systems (IDS) address IoT vulnerabilities by analyzing network traffic patterns and identifying anomalies as they occur. Deploying advanced machine learning-based IDS algorithms may detect real-time vulnerabilities and protect the IoT environment from malicious activity.

In the existing research, deep learning models are used for anomaly-based IDS. A transfer learning approach is

The associate editor coordinating the review of this manuscript and approving it for publication was Jiachen Yang¹.

frequently adopted, optimized through hyperparameter tuning. It achieved high accuracy but is constrained to major attack classes or binary classification, limiting its generalizability to broader IoT threats [1]. Some existing research on multi-class classification used a few attack classes rather than all the attack classes in the IDS datasets for developing an IDS system. The IDS system performed well for attack and benign classes but failed at macro-level minority classes classification, which was very poor [2]. The Fig. 1. illustrates an Intrusion Detection System (IDS) monitoring network traffic containing two types of DDoS attacks: a prevalent ICMP Flood and a less frequent HTTP Flood. This IDS model was trained on data where ICMP Flood samples significantly outnumbered HTTP Flood samples, which shows that IDS effectively identifies and mitigates major ICMP attacks. In contrast, its ability to detect the minor HTTP Flood attack is compromised, which showcases the challenge of data imbalance leading to poor generalization for underrepresented threats.

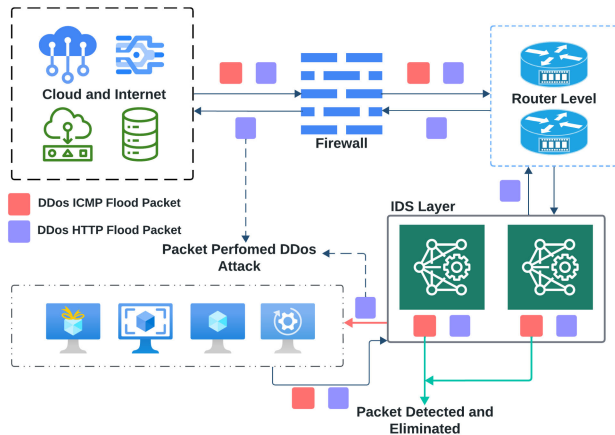


FIGURE 1. Impact of skewed training data: IDS effectively detects majority DDoS ICMP attacks (red) but misses minority DDoS HTTP attacks (purple) in live traffic.

To solve the data imbalance problem the existing research solutions applied synthetic data generation techniques using Generative Adversarial Networks (GAN), Magento, ADASYN, SMOTE, etc, to oversample the minority classes [3]. However, the dataset sizes are minimal to process the text IDS data, and these techniques utilize higher computational resources; at the same time, they are omitting sub-class classification and its imbalance issues; most of the *undersample* classes are not considered in the IDS, which creates severe security issues [4], [5]. To further solve the imbalanced problem, we have proposed work that transforms large datasets into 2D images for faster, resource-efficient training to minimize the duration of training and enhance the classification of various classes and their sub-classes. TIGAN applies optimized hybrid sampling to balance classes by generating synthetic data that helps to upsample the minority classes and downsample the majority classes. The TICNN model, trained on this balanced data, is evaluated with the *CICIoT2023*, *CICIDS2017* IDS datasets, covering multiple

attack class types and one benign class, achieving robust subclass-level anomaly detection.

Advanced machine/deep learning-based intrusion detection systems are proposed by many researchers, but there are still a lot of research issues which were not yet resolved:

- RQ1 While techniques like sampling [2], [6] and Generative Adversarial Networks (GANs) [7] are being actively explored to address data imbalance, how can we develop DL models that achieve both high accuracy and efficiency when classifying imbalanced data, particularly for rare or minority attack sub-classes [8]?
- RQ2 The synthetic data generation done by previous research has a synthetic bias. How can we design a sampling method and GAN that effectively generate correct amount of synthetic data?
- RQ3 With growing dataset sizes and model complexity, how can we ensure that DL-based IDS remain computationally efficient for real-time deployment without compromising classification accuracy, especially when dealing with high-dimensional data and multiple attack classes [9]?

The primary contributions of this work, which differentiate it from existing literature that often focuses on majority-class binary classification and high computational IDS models not feasible for existing scenarios, are as follows: (1) We propose a dense 2D RGB latent space transformation for tabular data that achieves a $4\times$ theoretical memory compression, addressing the memory bottlenecks of traditional IDS. (2) We introduce the TIGAN framework, which utilizes an adaptive IQR-based thresholding rule that helps mitigate the effects of synthetic bias associated with traditional GAN-based techniques of oversampling and undersampling RGB image data. (3) We develop the TICNN architecture. The lightweight model employs Global Average Pooling to achieve 1.81 million parameters while attaining 99.56% macro accuracy. In contrast to past efforts that relied solely on optimizing ensembles, we focus on optimizing both the granularity of detection for smaller minority subclasses and computational efficiency to facilitate IoT deployment at the edge of the network.

II. LITERATURE REVIEW

Researchers have developed anomaly-based Intrusion Detection Systems (IDS) that are robust and have success in attack detection in IoT networks. However, traditional IDS measures often fall short when faced with the unique challenges of IoT environments, such as device heterogeneity, resource limitations, and the sheer volume of data generated. Consequently, researchers are shifting more towards Machine Learning (ML) and Deep Learning (DL) techniques to develop more adaptive and effective IDS [9], [10].

A significant body of research focuses on applying various DL architectures to enhance IDS performance. Early work included the development of optimized Convolutional Neural Networks (CNNs) combined with Transfer Learning to secure the Internet of Vehicles (IoV) [1]. Transfer learning, which allows pre-trained models to be fine-tuned for specific

tasks, is beneficial in IoT. Concurrently, the adversarial aspect was also explored, with studies demonstrating how Conditional GANs (cGANs) can be used to craft attacks that deceive DL-based IDS in IoT networks [3].

To further this research, foundational enhancements were made, including the creation of new, large-scale datasets like CICIoT2023 to better represent real-time attacks [10]. Efforts also focused on generally enhancing AI-based NIDS using Generative Adversarial Networks (GANs) [4] and improving not just attack detection but also the explainability of model predictions in IoT systems [11], [12].

Building on these foundations, subsequent work has applied more sophisticated models. Researchers have demonstrated the combined power of CNN and Long Short-Term Memory (LSTM) networks for boosting IoT security using recent datasets [13], while further advancements in the IoV domain have included the use of tokenization alongside DL models to improve detection accuracy [14]. These studies collectively highlight a trend towards leveraging sophisticated deep learning models to build more accurate and context-aware intrusion detection systems [9].

Recent advancements have also focused on optimizing machine learning frameworks to handle the complexity of IoT network traffic. To address this, researchers created better machine learning frameworks based on their ability to handle high-dimensional data. These enhanced frameworks generally use PCA techniques in addition to some methods for reducing the number of features so that they perform better on a wide variety of tasks by removing unnecessary complexity from the feature-extraction process [15], [16]. Similarly, ensemble strategies have shown effective results; for instance, a tree-driven intelligent security system, which utilizes bagging ensembles to increase detection rates in diverse IoT environments [17]. To further improve upon the accuracy of these ensemble models, techniques based upon evolutionary optimization methods have been implemented. A perfect example of this is an ensemble learning approach using a butterfly optimization method to dynamically optimise weight parameters in order to enhance the strength and effectiveness of an intrusion detection system against different attack patterns [18]. Furthermore, recent studies have emphasized the importance of complex system security and intelligent detection in industrial contexts. For instance, research in industrial reliability engineering [19] and expert systems for security [20] highlights the need for robust anomaly detection in interconnected environments. Our proposed TICNN framework aligns with these trends by providing an efficient, image-based detection mechanism that is particularly suited for the high-stakes security requirements of industrial IoT (IIoT) systems.

However, a major problem in training effective IDS models is the inherent class imbalance in network traffic datasets, where malicious activities are often rare compared to benign traffic. This imbalance can bias models, leading to poor detection rates for critical, albeit infrequent, attacks. To counter this, various strategies have been explored. Traditional sampling techniques, such as SMOTE and ADASYN,

have been evaluated for their effectiveness in IoT datasets [6], and specialized methods like Weighted Optimized SMOTE (WOSMOT) have been integrated into DL pipelines for detecting specific attacks like DDoS [2].

GANs have become increasingly recognized as a powerful tool for generating high-quality synthetic minority-class data. Researchers have developed specialized GAN architectures like TMG-GAN [5] and CTGAN [9] specifically for imbalanced IDS datasets. Other work has utilized GANs for unsupervised anomaly detection, with models like FlowGANAnomaly learning network flow patterns to identify deviations [21]. In parallel, adversarial techniques have continued to evolve with novel methods like ProGen, which focuses on generating projection-based adversarial examples [22].

The latest research pushes the boundaries further with advanced hybrid models. These include approaches combining Swin Transformers and LSTM with transfer learning [23]. Furthermore, hybrid models combining Autoencoders (AE) with Wasserstein GANs (WGAN) have been proposed to both extract representative features and generate realistic synthetic attacks, achieving superior performance on established benchmarks like NSL-KDD and CIC-IDS2017 [7], [24].

This dual role of GANs fortifying defenses by generating synthetic data while simultaneously simulating advanced threats to probe for vulnerabilities positions them as a key technology in the ongoing cybersecurity arms race. The ability of GANs to learn complex data distributions, even demonstrated in fields like SAR-to-Optical image translation [25], underscores their potential impact on network security. However, the GAN techniques used can give rise to synthetic, biased training because the imbalance ratio among the classes is very high, a challenge that remains a focus of ongoing research.

III. PROPOSED FRAMEWORK

The proposed work first converts the dataset into images and then addresses the data imbalance issue using the proposed Tabular-Image Generative Adversarial Network (TIGAN) hybrid sampling method for image generation. This balanced dataset is used to train the proposed Transformed Image Convolution Neural Network (TICNN) model to achieve high results while maintaining performance on extensive IoT datasets.

A. DATASET PREPROCESSING AND IMAGE TRANSFORMATION THAT RESOLVES RQ3

The dataset consists of a substantial number of samples presented in one dimension that are represented using tables, which requires longer model training times and more computational power. To minimize both the training time and resource utilization at local client node, we convert the one-dimensional data into two-dimensional data represented in the form of RGB images.

Let's consider the dataset represented as $D \in \mathbb{R}^{M \times N}$, where each $d_{i,j}$ in D represents j^{th} feature value of i^{th} sample.

TABLE 1. Dataset details.

Dataset	No. of Samples (CSV)	No. of Samples (Image)	Image Size	No. of Classes	Feature Count	Imbalance Ratio	Preprocessing Time
CICIDS2017	2.5 Million	12.018K	70X70X3	15	70	9982:1	150 seconds
CICIoT2023	42 Million	27.618K	128X128X3	34	29	4251:1	1800 Seconds

$\mathbb{R}^{M \times N}$ is the original dataset with real-valued entries and $\mathbb{Z}^{M \times N}$ is the scaled dataset with integer entries representing RGB pixel values after normalization [26]. To normalize the feature values to the range suitable for RGB pixel values (0 to 255), we apply min-max scaling. The normalization function $S : \mathbb{R} \rightarrow [0, 255]$ is defined as Eq. No. 1, where D_j is the j -th column (feature) of D . After applying this transformation, we obtain a scaled dataset $\tilde{D} \in \mathbb{Z}^{M \times N}$.

$$S(d_{i,j}) = \frac{d_{i,j} - \min(D_j)}{\max(D_j) - \min(D_j)} \times 255 \quad (1)$$

Unlike prior image-based IDS approaches that merely map binary feature states or use grayscale representations, our approach transforms continuous tabular features into a dense 3D RGB latent space. This is fundamentally distinct because it preserves the scalar magnitude of attack features through channel-wise depth (R, G, B), allowing the CNN to learn inter-feature correlations as spatial textures rather than isolated values. Furthermore, by quantizing 32-bit floating-point tabular data into 8-bit integers (0-255 pixel values), we achieve a theoretical $4 \times$ memory compression. This explains the significant reduction in RAM usage observed in Figure 8, as the model processes compact tensor representations rather than sparse, high-dimensional tabular vectors.

To produce an RGB image, it is necessary to reshape the transformed data $\tilde{D}$ where R represents the total number of rows in the image and C signifies the total number of columns across the Red, Green and Blue color channels, respectively. This transformation produces a structure output based on Algorithm 1, resulting in an image dataset termed as $\mathbb{I}^{R \times C \times 3}$. Table 1 represents the dataset details generated using Algorithm 1.

Algorithm 1 Image Generation Algorithm

Input: $V = \{v_1, v_2, \dots, v_n\}$ — Sequence of pixel vectors.

$P \in \mathbb{Z}^+$ — Required number of pixels.

$Z \in \mathbb{Z}^+$ — Number of padding pixels.

Output: $I \in \mathbb{I}^{R \times C \times 3}$ — Generated image tensor.

```

1:  $N \leftarrow |V|$  // Number of available pixel vectors
2: if  $N \geq P$  then
3:    $I \leftarrow \text{reshape}(V, (R, C, 3))$ 
4: else
5:    $Z \leftarrow P - N$ 
6:    $V_{\text{zero}} \leftarrow \mathbf{0}^Z$  // Zero-padding vector
7:    $V \leftarrow V \cup V_{\text{zero}}$ 
8:    $I \leftarrow \text{reshape}(V, (R, C, 3))$ 
9: end if
10: return  $I$ 

```

B. PROPOSED TIGAN AND HYBRID-SAMPLING FRAMEWORK THAT RESOLVED RQ2

Various dataset-balancing techniques and algorithms are used to generate synthetic data. Two primary approaches for handling class imbalance in tabular data are oversampling and undersampling. However, these methods come with significant limitations in highly imbalanced scenarios. In the case of oversampling, especially when the minority-to-majority class ratio is extremely low, the IDS (Intrusion Detection System) model tends to become synthetic-biased—favoring the artificially generated samples over real ones, which compromises generalization and detection accuracy. On the other hand, undersampling in such cases can result in up to 99% data loss, discarding valuable information from the majority class and degrading model performance. To address this issue, we proposed a hybrid sampling algorithm that finds a threshold value and generates new images or removes excess image samples by defining a threshold value. To determine a suitable threshold T for class balancing in our dataset, we calculate several key statistical measures across the class distributions. Let $C = \{c_1, c_2, \dots, c_n\}$ represent the set of classes, each with an image count $N(c_i)$ for class c_i .

First, we calculate the **median** m of the image counts, which serves as a central measure of our class distribution. The median m is formulated as:

$$m = \begin{cases} N\left(c_{\frac{n}{2}}\right) & \text{if } n \text{ is odd} \\ \frac{N\left(c_{\frac{n}{2}-1}\right) + N\left(c_{\frac{n}{2}}\right)}{2} & \text{if } n \text{ is even} \end{cases} \quad (2)$$

In addition, we compute the 25th percentile Q_1 and the 75th percentile Q_3 of the image counts vector $\mathbf{N} = [N(c_1), N(c_2), \dots, N(c_n)]^T$, which describe the lower and upper quartiles, respectively. These are essential for calculating the **interquartile range (IQR)** [27], which is given by:

$$IQR = Q_3 - Q_1 \quad (3)$$

The **upper bound for outliers** is then defined as $Q_3 + 1.5 \cdot IQR$, allowing us to set an alternative threshold that considers high outlier values. Specifically, the threshold T can be set based on the distribution characteristics, as follows:

$$T = \begin{cases} m & \text{(using Median Count)} \\ Q_3 + 1.5 \cdot IQR & \text{(to capture High Outliers)} \end{cases} \quad (4)$$

Working Example of IQR Thresholding: To illustrate the application of Equation 4, consider an IoT dataset where the sample counts for five attack classes are $\{10, 20, 50, 100, 500\}$. Here, the quartiles are $Q_1 = 20$ and $Q_3 = 100$, resulting in an *IQR* of 80. Applying the

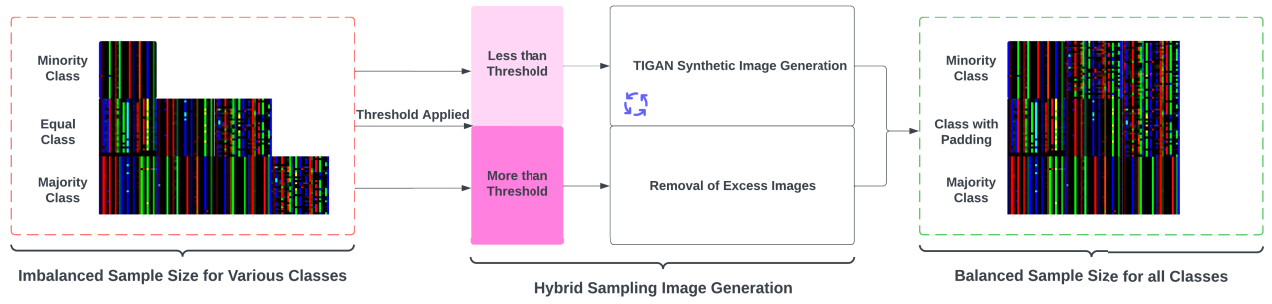


FIGURE 2. Proposed TIGAN framework illustrating a hybrid sampling technique for addressing class imbalance. The process begins with imbalanced classes, applies a threshold to differentiate between minority and majority classes, then uses TIGAN (Threshold-Integrated Generative Adversarial Network) for synthetic image generation to augment minority classes and removes excess images from majority classes, ultimately achieving a balanced sample size across all classes.

high-outlier threshold $T = Q_3 + 1.5 \times IQR$ yields $T = 100 + 1.5(80) = 220$. Consequently, TIGAN will synthesize 210 and 200 new samples for the first two classes respectively, and downsample the majority class from 500 to 220. This ensures the model is not overwhelmed by the majority class while providing sufficient representative data for minority threats.

This approach allows us to balance the dataset by either targeting the median class count or adjusting for outliers, providing flexibility based on the specific distribution of the class image counts.

This hybrid sampling strategy provides a significant practical advantage: it maintains the diversity of the majority class through intelligent downsampling while aggressively augmenting sporadic minority classes. By operating in the image domain, TIGAN avoids the computational *out-of-memory* errors typically encountered when applying SMOTE or ADASYN to massive 42-million-record tabular datasets that may scale it up to more than 100 million records, which makes it more difficult to process.

Algorithm 2 Class Balancing Using Hybrid-Sampling

Input: Class set $C = \{c_1, c_2, \dots, c_n\}$, threshold T .

Output: Balanced dataset with each class having exactly T images.

```

1: for each class  $c_i \in C$  do
2:   if  $N(c_i) < T$  then
3:     Augment the class using TIGAN:
4:      $TIGAN(c_i) \rightarrow$  Generate images for  $c_i$ 
5:   else
6:     Delete excess images to ensure  $T$  images:
7:      $N(c_i) \leftarrow T$ 
8:   end if
9: end for
10: return Balanced dataset with all classes  $C$  containing exactly  $T$  images.

```

In the TIGAN framework given in Fig. 3. and Fig. 4., involves two main components that are the generator g and the discriminator d . The discriminator's goal is to differentiate between authentic images and those that are generated

(fake) [25]. This objective is mathematically expressed through a loss function, $\mathcal{J}_{\text{disc}}$, which is formulated as follows:

$$\mathcal{J}_{\text{disc}} = \mathbb{E}_{x \sim p_{\text{real}}(x)} [\log d(x)] + \mathbb{E}_{s \sim p_{\text{latent}}(s)} [\log (1 - d(g(s, y)))] \quad (5)$$

Here, the equation $\mathbb{E}_{x \sim p_{\text{data}}(x)} [\log d(x)]$, shows the expected logarithmic probability that the discriminator correctly classifies real images from the data distribution $p_{\text{data}}(x)$. The other equation, $\mathbb{E}_{s \sim p_{\text{latent}}(s)} [\log (1 - d(G(s, y)))]$, reflects the expected logarithmic probability that the discriminator correctly identifies generated images (produced by the generator using a random noise vector z and class label y) as fake. The discriminator uses these two components, leading to a final loss defined as $-\mathcal{J}_{\text{disc}}$. During training, the discriminator's weights are updated to maximize this loss through back-propagation, enhancing its ability to correctly classify real and fake images. Conversely, the generator aims to produce images that the discriminator classifies as real. This objective is represented by the generator's loss function, $\mathcal{J}_{\text{gen}}$, which is defined as:

$$\mathcal{J}_{\text{gen}} = \mathbb{E}_{s \sim p_{\text{latent}}(s)} [\log f(g(s, y))] \quad (6)$$

This expression captures the expected log probability that the discriminator assigns a high probability to the images generated by g . To facilitate training, the generator aims to minimize this loss, which can also be represented as $-\mathcal{J}_{\text{gen}}$. This adversarial setup leads to a dynamic where both networks continuously improve, resulting in the generation of high-quality synthetic images conditioned on specified class labels. The technique does oversampling for the major classes and under-sampling for minor classes based on a determined threshold, which balances the data and is trained on the proposed TICNN model.

Distinct from tabular-specific GANs like CTGAN [9] or TMG-GAN [5], which model column distributions using Gaussian mixtures, TIGAN operates directly in the transformed image domain. This allows the generator to use transposed convolutions to synthesize complex feature dependencies spatially, avoiding the mode collapse often seen in tabular GANs when dealing with high-cardinality features. Regarding the balancing threshold T , unlike static sampling ratios used in SMOTE, our approach dynamically

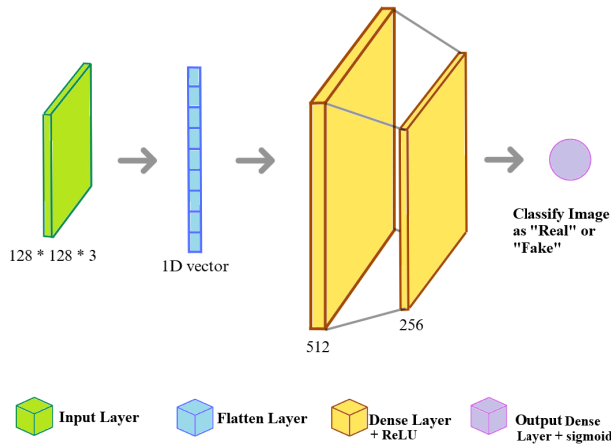


FIGURE 3. Proposed TIGAN discriminator architecture.

selects T based on the Interquartile Range (Equation 4). This ensures the threshold adapts to the skewness of the specific dataset distribution, preventing over-generation in slightly imbalanced classes while aggressively augmenting extreme minority classes.

C. PROPOSED TRANSFORMED IMAGE CONVOLUTION NEURAL NETWORK (TICNN) THAT RESOLVES RQ1

The proposed model for image classification consists of convolutional layers with 64, 128, and 256 filters using ReLU activation to extract features. Max pooling layers downsample feature maps, maintaining important details. A Global Average Pooling layer reduces parameters, followed by a fully connected layer consisting of 256 units paired with a dropout layer set at a 0.5 rate to mitigate overfitting. The model employs softmax activation along with categorical cross-entropy loss to enhance classification performance and optimization, with each component serving a specific purpose. The Table 2 represents the TICNN architecture model.

TABLE 2. CNN architecture for TICNN.

Layer	Type	Filters/Units	Kernel Size	Strides	Activation
1	Conv2D	64	(3, 3)	(1, 1)	ReLU
2	Conv2D	64	(3, 3)	(1, 1)	ReLU
3	MaxPooling2D	-	(2, 2)	-	-
4	Conv2D	128	(3, 3)	(1, 1)	ReLU
5	Conv2D	128	(3, 3)	(1, 1)	ReLU
6	MaxPooling2D	-	(2, 2)	-	-
7	Conv2D	256	(3, 3)	(1, 1)	ReLU
8	Conv2D	256	(3, 3)	(1, 1)	ReLU
9	Conv2D	256	(3, 3)	(1, 1)	ReLU
10	GlobalAveragePooling2D	-	-	-	-
11	Dense	256	-	-	ReLU
12	Dropout	rate=0.5	-	-	-
13	Dense	num_class	-	-	Softmax

The 2D convolution operation is a key element, applied to an input image, denoted I , using a kernel K , and it yields an output feature map O . For each position (m, n) in the output, the feature value $O(m, n)$ is computed by summing element-wise products between the kernel and corresponding portions of the image, mathematically expressed as:

$$O(m, n) = \sum_{p=0}^{P-1} \sum_{q=0}^{Q-1} K(p, q) \cdot I(m+p, n+q) \quad (7)$$

Here, $O(m, n)$ denotes the computed feature at position (m, n) , representing a localized interpretation of the input data, allowing the network to capture spatial patterns. The input image I measures H by W , with H representing its height and W its width. The convolution kernel K is a matrix of dimensions $P \times Q$, where P and Q define the height and width of the filter. Indices p and q iterate across the kernel's dimensions, aggregating information from all kernel locations to build each feature map component. For classification, the TICNN model uses softmax activation in the final layer to convert raw output logits into probabilities for each class [1]. Given the logits z from the model's last layer, the softmax function calculates the predicted probability $\hat{y}_i$ for class i as follows:

$$\hat{y}_i = \sigma(z_i) = \frac{e^{z_i}}{\sum_{j=1}^C e^{z_j}} \quad \text{for } i = 1, \dots, C \quad (8)$$

In this equation, $\hat{y}_i$ represents the predicted probability for class i , and z_i is the corresponding logit for that class. The softmax function transforms these logits so that they sum to 1 across all C classes, ensuring each output is interpretable as a probability. This probability distribution is beneficial in multi-class classification tasks as it allows the network to assign confidence levels across multiple categories. To evaluate model performance, the TICNN employs the categorical cross-entropy loss function [1], which measures the difference between the true class distribution y and the predicted probabilities $\hat{y}$. This loss function is defined as:

$$\mathcal{L}(y, \tilde{y}) = - \sum_{k=1}^K y_k \log(\tilde{y}_k) \quad (9)$$

In this scenario, $\mathcal{L}(y, \tilde{y})$ represents the loss for a prediction. The binary indicator y_k indicates if class k is correct, while $\tilde{y}$ shows the predicted probability for that class. Categorical cross-entropy loss is useful in multi-class classification as it penalizes high-confidence wrong predictions, helping the model increase the probability of the correct class and decrease it for incorrect ones.

IV. PERFORMANCE EVALUATION AND RESULTS

The experiments were conducted on a system equipped with an AMD Ryzen Threadripper Pro processor featuring 12 cores, 62.7 GiB of memory, and an NVIDIA GeForce RTX 3070 GPU. The primary dataset used for evaluation is the CICIOT2023 dataset, which contains over 46 million attack samples distributed across 33 attack classes and 1 benign class, as visualized in Fig. 6. The dataset is highly imbalanced, posing a substantial challenge for accurate and fair classification. This makes it well-suited for evaluating the robustness and effectiveness of deep learning-based intrusion detection systems. To further assess the generalizability of the proposed model, the CICIDS2017 dataset was also employed. CICIDS2017 comprises 14 attack classes and 1 benign class represented in Fig. 7, offering a broad spectrum of contemporary network threats including DDoS, PortScan, Botnet, and more. The inclusion of both datasets—each

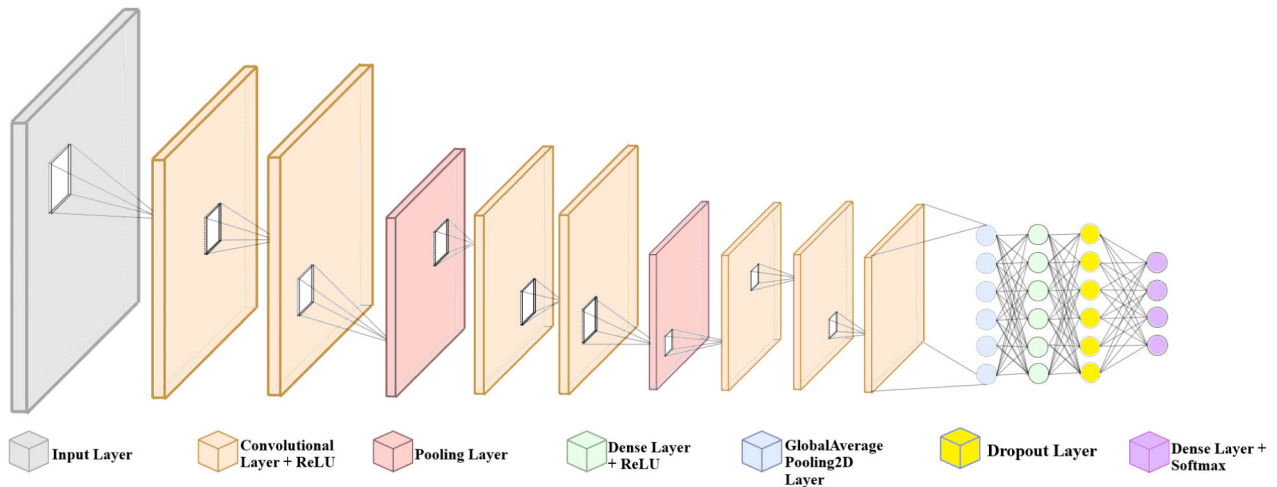


FIGURE 4. Proposed TICNN architecture showing the flow from the input layer through convolutional, pooling, dense, dropout, and softmax layers used for classification tasks.

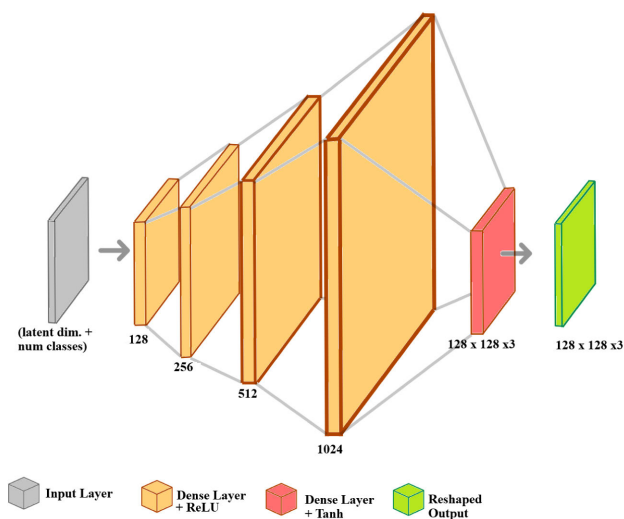


FIGURE 5. Proposed TIGAN generator architecture.

with distinct characteristics and class distributions—enables a comprehensive evaluation of the model’s performance in both IoT-specific and general network environments.

A. COMPUTATIONAL RESOURCES UTILIZATION

The Fig. 8. presents a comparative analysis of RAM utilization and time taken for loading data in the CICIOT2023, CICSIDS2017 dataset across five training epochs using two different formats: CSV (tabular data) and image-based data. The blue line indicates RAM consumption for single dimension data represented in form of comma separated value loading, which exceeds 40 GB and continues to rise, reaching over 44 GB by the 5th epoch. Simultaneously, the red dashed line shows that the time taken for CSV data loading approaches 7200 seconds (approximately 2 hours), revealing its inefficiency in handling large datasets. In comparison with single dimensional data the two dimensional data represented in the form of images performed well where the orange and

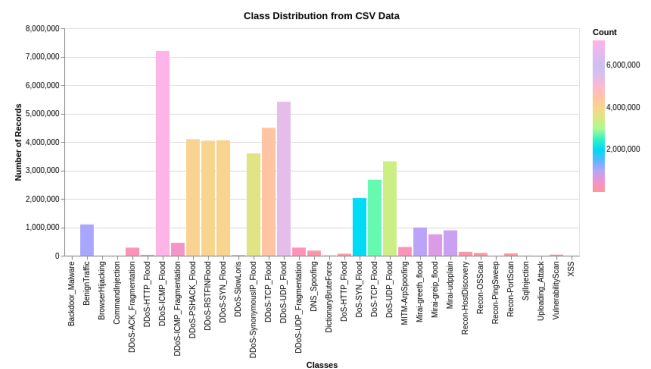


FIGURE 6. CICIoT2023 dataset visualization.

green lines represent the RAM usage and time taken for image data loading. RAM utilization remains under 8 GB across all epochs, and the time taken stays consistent at around 30 seconds, making this approach highly efficient in terms of both memory and speed that is leading to faster training times and reduced memory overhead.

B. TIGAN TRAINING AND BALANCED DATA

The TIGAN model was trained for 5000–10000 epochs based on different classes taking 360–800 seconds of training time for each minority sub-class. Figure 9 depicts the loss graph for discriminator and generator on subclasses. The TIGAN model was trained on 33 distinct subclasses using hybrid sampling techniques to enhance image generation quality, producing 170 threshold images for each class. A pie chart, Fig. 10, visualises the distribution of these images, highlighting class representation and revealing insights into performance and areas needing refinement. The results confirm the successful application of the TIGAN model in generating relevant images. Overall, these findings underscore TIGAN’s potential to create high-quality, subclass-specific images and contribute significantly to research in synthetic image generation.

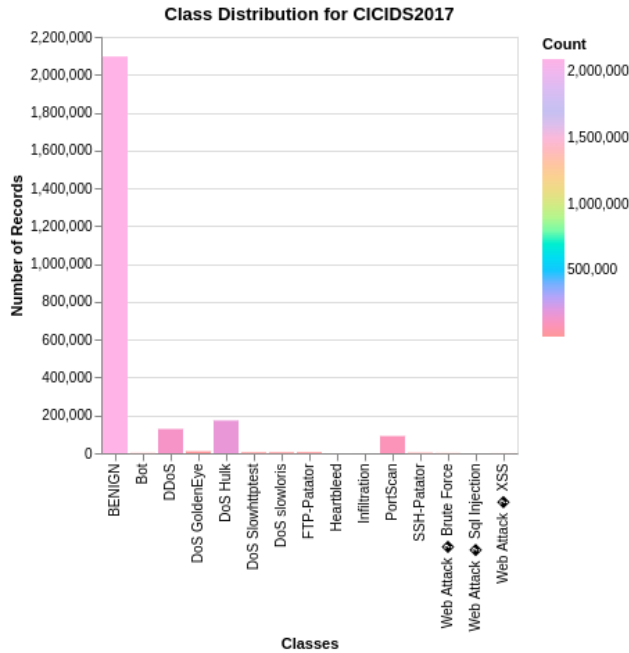


FIGURE 7. CICIDS2017 dataset visualization.

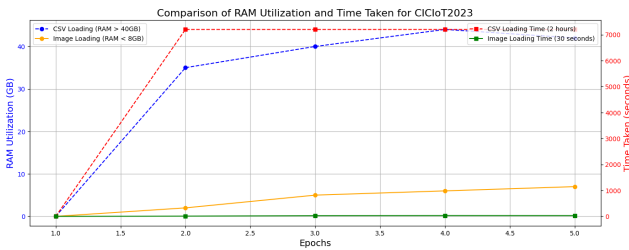


FIGURE 8. Computational resource utilization on image VS CSV data.

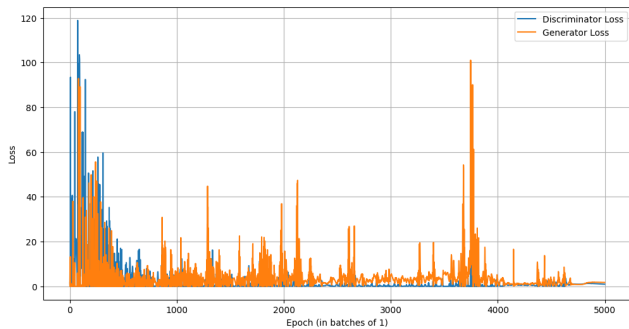


FIGURE 9. Loss curves of the TIGAN model over 5000 training epochs. The plot shows the discriminator and generator losses. Early stages show high fluctuation, but both losses gradually stabilize.

C. TICNN RESULT ANALYSIS

The TICNN model was trained maximum of up to 250 epochs for CICIDS2017 and 100 epochs for CICIoT2023. Training time for each epoch was less than 5 seconds after data balancing using the TIGAN model.

The **Weighted Average** takes into account the number of actual instances in each class when computing metrics, ensuring that classes with more instances have a greater impact on the final average [3]. It is calculated using the

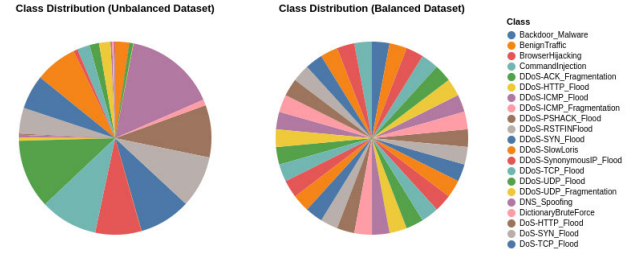


FIGURE 10. Impact of hybrid sampling technique and TIGAN on class distribution. The original imbalanced dataset (left) is transformed into a balanced dataset (right) with more equitable representation across all classes.

equation 10 where S_i represents the support for class i and M_i denotes the metric (precision, recall, or F1-score) for that class. This method is especially useful for imbalanced datasets.

$$\text{Weighted Avg Metric} = \frac{\sum_{i=1}^n (S_i \cdot M_i)}{\sum_{i=1}^n S_i} \quad (10)$$

In contrast, the **Macro Average** calculates the metric for each class independently and then computes the unweighted average. This method considers all classes on an equal footing, which makes it appropriate for assessing model performance across all classes, irrespective of their size.

$$\text{Macro Avg Metric} = \frac{1}{n} \sum_{i=1}^n M_i \quad (11)$$

In summary, the **Weighted Average** is ideal for uneven class distributions, while the **Macro Average** is better for equal consideration of all classes.

The results in TABLE 3 compare the performance of the models VGG16, VGG19, Xception, and TICNN on imbalanced and balanced dataset for CICIoT2023. For the imbalanced dataset, all models exhibited high accuracy, but Xception had the lowest macro average precision at 0.722. VGG19 and VGG16 followed with scores of 0.784 and 0.765, respectively. In contrast, the TICNN model significantly outperformed the others across all weighted average metrics, achieving a precision, recall, and F1-score of 0.998. Its macro average precision was 0.855, the highest among the models, indicating its effectiveness in classifying minority instances despite the imbalance.

When using a balanced dataset, all models showed improved performance. The VGG16, VGG19, and Xception models demonstrated better uniformity in performance, achieving macro average scores of 0.970, 0.967, and 0.959, respectively. The TICNN model maintained its lead with a weighted average precision of 0.997, recall of 0.995, and F1-score of 0.996. Its macro average precision and recall also improved to 0.994 and 0.995, respectively. Packet detection times for CICIoT2023 ranged from 1.5 ms to 2.92 ms.

The results presented in TABLE 4 detail the performance evaluation of the VGG16, VGG19, Xception, and TICNN models, specifically on the CICIDS2017 dataset, mirroring the comparative structure used for CICIoT2023. The table contrasts the models' effectiveness before and after applying

TABLE 3. Performance comparison of VGG16, VGG19, Xception, and TICNN models before and after performing TIGAN for CICIOT2023 (TICNN Enhanced).

Metric	Before Adversarial Generation				After Adversarial Generation			
	VGG16	VGG19	Xception	TICNN	VGG16	VGG19	Xception	TICNN
Weighted Avg Precision	0.994	0.995	0.987	0.998	0.970	0.969	0.959	0.997
Weighted Avg Recall	0.995	0.995	0.988	0.998	0.960	0.969	0.955	0.995
Weighted Avg F1-Score	0.994	0.995	0.987	0.998	0.961	0.969	0.956	0.996
Macro Avg Precision	0.765	0.784	0.722	0.855	0.970	0.967	0.959	0.995
Macro Avg Recall	0.760	0.785	0.730	0.846	0.960	0.967	0.955	0.994
Macro Avg F1-Score	0.758	0.778	0.722	0.846	0.961	0.967	0.956	0.994
Overall Accuracy	0.995	0.995	0.998	0.998	0.960	0.969	0.955	0.997
Packet Detection time (in ms)	1.5-2.92							

TIGAN for adversarial generation, aiming to create a balanced dataset. On the imbalanced CICIDS2017 dataset ('Before Adversarial Generation'), all models achieved substantial weighted average precision, recall, and F1-scores, along with overall accuracy, primarily in the 0.984 to 0.988 range. This indicates good performance, likely dominated by the majority classes. However, the macro average metrics that helps us to understand macro level sample classes were considerably lower, ranging from 0.622 (Xception's Macro Precision and F1-Score) to 0.755 (TICNN's Macro Precision). This disparity underscores the challenge posed by imbalance, with TICNN (highlighted in blue) showing a clear advantage in handling these conditions compared to the other models.

There were notable shifts when evaluated on the balanced CICIDS2017 dataset ('After Adversarial Generation'). While VGG16, VGG19, and Xception achieved high weighted average precision, recall, and overall accuracy in the 0.980 to 0.989 range, their F1-scores and macro average scores were lower, generally falling between 0.915 and 0.930. In stark contrast, the TICNN model (highlighted in dark green and bold) demonstrated exceptional performance, achieving scores between 0.991 and 0.992 across all its 'After TIGAN' metrics (Weighted, Macro, and Accuracy). This reinforces TICNN's robustness and superior classification capability, especially when trained on the TIGAN-balanced dataset. Furthermore, the Packet Detection time for the CICIDS2017 dataset is reported as being within the 3 to 3.5 ms range, which is slightly higher than previously noted for CICIOT2023. Overall, TABLE 4 indicates that for the CICIDS2017 dataset, TIGAN helps improve performance, but the TICNN model stands out as significantly more effective, reaching near-optimal levels after balancing.

The TABLE 5 compares the performance of a classification model across different types of cyber-attacks under two data conditions: imbalanced and balanced. Key metrics like precision, recall, and F1 score reveal the model's accuracy in identifying attack classes. Precision reflects the correctness of identified attacks, while recall measures the detection of actual attacks. The F1 score balances both metrics.

In the imbalanced data scenario, attack classes like Browser Hijacking, Recon-Ping Sweep, and XSS showed very low performance, with precision, recall, and F1 scores near zero, indicating frequent misclassification and rare detection due to under-representation. This scarcity led the model to focus on more common attack types, resulting

in bias and significant misclassification of these minority classes. However, when the data was balanced, performance improved substantially across most attack types, particularly in recall. For instance, Backdoor Malware saw its recall increase from 0.500 to 0.999, while Browser Hijacking and Recon-Ping Sweep, previously undetected, improved to recalls of 0.962 and 0.961, respectively. Balancing the data enabled the model to better identify these attacks by providing a more representative sample for each class and reducing bias toward majority classes. In the imbalanced data scenario, certain attack classes like *Browser Hijacking*, *Recon-Ping Sweep*, and *XSS* show very low performance, with scores near zero across precision, recall, and F1 metrics. This suggests that these attacks were often misclassified or rarely detected, likely due to their under-representation in the imbalanced dataset. This scarcity led the model to focus on more frequent attack types, resulting in a strong bias and significant misclassification of these minority classes. When the data was balanced, performance on most attack types improved substantially, especially in recall, indicating a higher rate of correctly identified attacks across classes. For instance, *Backdoor Malware* improved in recall from 0.500 to 0.999, while *Browser Hijacking* and *Recon-Ping Sweep*, previously undetected (with recalls of 0.000), saw a recall boost to 0.962 and 0.961, respectively. Balancing the data helped the model better identify these attacks by providing a more representative sample for each class, reducing its bias toward majority classes and enabling it to generalize more effectively.

TABLE 7 shows our TICNN model achieves accuracy comparable to state-of-the-art methods [5], [12], [23], [24] on CICIDS2017 and CICIOT2023 datasets. While other models reach high accuracy, often through heavy synthetic data generation that might introduce bias [5], [24], our approach focuses on efficiency. By transforming data into 2D images, TICNN significantly reduces both computational and detection times. This allows our model to deliver top-tier accuracy with lower resource demands. Consequently, TICNN offers a more practical and efficient solution for real-time intrusion detection systems.

To ensure the statistical reliability of these results, we performed 5 independent runs using Stratified K-Fold Cross-Validation in Section IV-D.3. The reported accuracy of 99.56% represents the mean performance, with a negligible standard deviation ($\sigma < 0.05\%$) observed across folds, confirming the model's stability. Furthermore, in terms

TABLE 4. Performance comparison of VGG16, VGG19, Xception, and TICNN models before and after performing TIGAN for CICIDS2017 (TICNN Enhanced).

Metric	Before Adversarial Generation				After Adversarial Generation			
	VGG16	VGG19	Xception	TICNN	VGG16	VGG19	Xception	TICNN
Weighted Avg Precision	0.984	0.985	0.987	0.988	0.981	0.985	0.988	0.992
Weighted Avg Recall	0.985	0.985	0.988	0.988	0.980	0.989	0.985	0.991
Weighted Avg F1-Score	0.984	0.985	0.987	0.988	0.921	0.929	0.916	0.991
Macro Avg Precision	0.665	0.684	0.622	0.755	0.930	0.927	0.919	0.992
Macro Avg Recall	0.660	0.685	0.630	0.746	0.920	0.927	0.915	0.995
Macro Avg F1-Score	0.658	0.678	0.622	0.746	0.921	0.927	0.916	0.997
Overall Accuracy	0.985	0.985	0.988	0.988	0.980	0.989	0.985	0.996
Packet Detection time (in ms)	3-3.5							

TABLE 5. Comparison of major and minor class attacks metrics before and after performing TIGAN for CICIOT2023 and CICIDS2017.

Attack Type	Minor Classes	Before Adversarial Generation			After Adversarial Generation		
		Precision	Recall	F1 Score	Precision	Recall	F1 Score
Malware	Backdoor Malware	0.500	0.500	0.500	0.998	0.999	0.999
	Browser Hijacking	0.000	0.000	0.000	1.000	0.962	0.984
	Command Injection	0.600	0.300	0.400	0.998	0.997	0.998
	SQL Injection	0.692	0.900	0.783	0.999	0.998	0.999
DDoS Attacks	DDoS-UDP Flood	0.998	0.999	0.999	1.000	1.000	1.000
	DDoS-HTTP Flood	0.750	1.000	0.854	0.999	0.931	0.963
	DDoS-SynonymousIP_Flood	0.993	0.996	0.996	0.999	0.998	0.999
Reconnaissance	Recon-Ping Sweep	0.000	0.000	0.000	0.998	0.961	0.980
Brute Force	Dictionary Brute Force	0.724	0.781	0.808	1.000	0.884	0.934
Cross Site Scripting	XSS	0.000	0.000	0.000	1.000	0.968	0.981
DNS	DNS Spoofing	0.866	0.831	0.821	0.958	0.981	0.971

of computational latency, TICNN demonstrates a distinct advantage. While baseline models like VGG16 and Xception typically exhibit inference times ranging from 12 ms to 22 ms due to their deep architectures and dense parameter counts, TICNN achieves a packet detection time of just 1.5–2.9 ms. This approx. $6\times$ speedup validates the efficiency of replacing heavy fully connected layers with Global Average Pooling, making TICNN highly suitable for real-time IoT constraints.

To rigorously quantify this efficiency, we analyzed the computational complexity of TICNN in comparison to the baseline architectures. Table 6 details the Parameter count (Params), Floating Point Operations (FLOPs), and architectural depth of each model. While VGG16 and VGG19 rely on massive parameter sets (approx. 138M and 143M, respectively) largely due to their dense fully connected headers, TICNN utilizes a compact design with only 1.81 million parameters. This represents a model size reduction of over 98% compared to VGG-based baselines. Furthermore, TICNN requires only 6.1 GFLOPs per inference, significantly lower than the 30.9 GFLOPs required by VGG16. This reduction in computational load, combined with a shallow depth of 13 layers (vs. 71 for Xception), directly correlates to the observed low-latency performance, making TICNN uniquely suited for resource-constrained IoT edge devices.

D. ABLATION STUDY

1) IMAGE GENERATION

This ablation study examines the effects of image resolution and feature dimensionality on model performance in image generation tasks. By segmenting the dataset based on feature

TABLE 6. Computational complexity comparison: TICNN vs. Baselines.

Model	Params (M)	FLOPs (G)	Depth (Layers)	Latency (ms)
VGG16	≈ 138.3	≈ 30.9	16	12 – 15
VGG19	≈ 143.6	≈ 39.0	19	14 – 17
Xception	≈ 22.9	≈ 8.4	71	18 – 22
TICNN (Ours)	1.81	6.10	13	1.5 – 2.9

TABLE 7. Comparison of CICIDS2017 and CICIOT2023 with state of the art for multiclass classification and TICNN - overall accuracy.

Dataset	Overall Accuracy
CICIOT2023 [23]	0.974
CICIOT2023 [12]	0.991
CICIDS2017 [5]	0.996
CICIDS2017 [24]	0.990
CICIOT2023 (Ours)	0.996
CICIDS2017 (Ours)	0.998

size, we explore the trade-offs among macro-averaged performance metrics, overfitting tendencies, data fidelity, and preprocessing duration. The findings indicate that the image generation algorithm demonstrates resilience to variations in image size, suggesting its capability to accommodate diverse model architectures effectively.

The number of samples generated is calculated as:

$$\text{No. of Samples} = \left\lceil \frac{\text{Image Size}}{\text{Feature Size}} \right\rceil \quad (12)$$

where the image size is represented as Height $\times$ Width $\times$ Channels, and the feature size corresponds to the model's receptive field or layer output size.

We experimented with six common image sizes: $29 \times 29 \times 3$, $71 \times 71 \times 3$, $128 \times 128 \times 3$, $224 \times 224 \times 3$, $256 \times 256 \times 3$ for

CICIoT2023 and added $70 \times 70 \times 3$ for CICIDS2017. The results, including macro-averaged performance, overfitting potential, data loss percentage, and preprocessing time for each image size, are presented in TABLE 8.

From the results, the following trends were observed:

- **Macro Avg Performance:** Performance variations are minimal at a macro level; however, detailed analysis at the subclass level reveals significant disparities. The quantity of images used in deep learning is crucial, as a sufficient dataset size is essential. As image dimensions increase, the number of viable samples correspondingly decreases, which can impact model training and generalization. The proposed image generation algorithm is robust to image size in terms of model architecture but creating images based on feature size will lead to no data loss and will always give better results.
- **Prone to Overfitting:** The number of samples required varies depending on the specific model. Our analysis indicates that insufficient samples can lead to overfitting. Additionally, larger image sizes result in reduced effective data, further increasing the risk of overfitting.
- **Data Loss:** Data loss increases as the image size increases, with the lowest data loss observed for feature size-based images.
- **Preprocessing Time:** The image generation algorithm demonstrates resilience to varying preprocessing durations, regardless of whether the input is sourced from CSV data or directly from images. This robustness stems from the necessity to accommodate different neural network architectures, each requiring images of specific dimensions. Rescaling images often results in significant data loss, making it crucial to generate images that meet the required size specifications from the outset. Our approach minimizes data loss to negligible levels and ensures that preprocessing times remain consistent, irrespective of the target image size.

This study concludes the critical role of selecting an optimal image size in relation to feature dimensions and the specific requirements of the model. It is established that image sizes tailored to feature size consistently yield superior performance. Nonetheless, certain CNN architectures demand larger image sizes, making feature-size-based dimensions unsuitable in those contexts. In such scenarios, a generalized image size approach is warranted. The research was able to develop a robust algorithm capable of delivering equivalent results across varying image sizes, thus ensuring consistency and reliability irrespective of the chosen input dimensions.

2) HYBRID SAMPLING TECHNIQUE AND TIGAN ANALYSIS

Oversampling and undersampling are established methodologies for mitigating class imbalance in datasets. Oversampling techniques, such as Random Oversampling, SMOTE (Synthetic Minority Oversampling Technique), and ADASYN, aim to augment the instances of the minority class to achieve a more balanced dataset. Although oversampling can enhance model performance in scenarios of imbalance, it carries the

risk of overfitting, particularly with random oversampling, as it merely replicates existing minority instances, potentially leading the model to be overly attuned to specific samples. Conversely, undersampling involves the reduction of majority class instances through approaches like Random Undersampling, Tomek Links, and NearMiss. This method can streamline the dataset and expedite training times but may inadvertently discard vital information concerning the majority class, which could detrimentally impact the model's ability to generalize. Hybrid sampling strategies, which integrate both oversampling and undersampling techniques, offer a compelling alternative by capitalizing on the strengths of both approaches while mitigating their respective weaknesses. By generating synthetic samples for the minority class while concurrently minimizing redundancy in the majority class, hybrid methods preserve the dataset's diversity. This balance aids in reducing both overfitting and underfitting risks, ultimately making hybrid sampling more potent for enhancing model performance on imbalanced datasets while safeguarding data integrity and generalization capabilities.

Analysis: SMOTE is simpler to use but more prone to overfit in sparse regions; both ADASYN and SMOTE perform well with tabular data. Although ADASYN has severe imbalances, it adapts to sample density. In every scenario, including photographs, TIGAN works remarkably effectively by combining over- and under-sampling. Its hybrid design allows it to handle different data distributions while minimizing overfitting and preserving the majority class's diversity. TIGAN is a dependable solution for unbalanced datasets since it can generate an arbitrary amount of synthetic images, even though it requires more processing power but utilizes less memory as compare to ADASYN and SMOTE.

The results presented in Table 10 indicate that under-sampling as a strategy does not yield effective performance in the context of the datasets evaluated (CICIoT2023 and CICIDS2017). For both datasets, the accuracy obtained through undersampling is extremely low, with values of 0.02% for CICIoT2023 and 0.00% for CICIDS2017. This is in stark contrast to the higher accuracy obtained with oversampling and hybrid sampling strategies, which produced 96.5% and 98.0% accuracy for the CICIoT2023 and CICIDS2017 datasets, respectively. Undersampling reduces the amount of data available for training by eliminating minority class samples, which can lead to a significant loss of crucial information and account for its subpar performance. When there is a large class imbalance, undersampling exacerbates the imbalance problem and reduces the model's capacity to generalize. This is shown by the noticeably lower accuracy rates for undersampling in both datasets. However, oversampling and hybrid sampling techniques help to balance the class distribution and improve performance while maintaining the model's ability to learn from both majority and minority classes. Despite being slightly more precise than oversampling, hybrid sampling requires less processing power and in this case it performs well because imbalanced ratio is high.

TABLE 8. Combined performance metrics across image sizes for CICIDS2017 and CICIOT2023.

Dataset	Image Size	Macro Avg (%)	Prone to Overfitting	Data Loss (%)	Preprocessing Time (s)
CICIDS2017	$70 \times 70 \times 3$	98.88	No	0.0000	150
	$71 \times 71 \times 3$	98.87	No	0.0000	150
	$128 \times 128 \times 3$	98.00	No	0.0014	150
	$224 \times 224 \times 3$	98.00	Yes	0.0002	150
	$256 \times 256 \times 3$	97.55	Yes	0.0018	150
CICIOT2023	$29 \times 29 \times 3$	97.11	No	0.0000	1800
	$71 \times 71 \times 3$	97.00	No	0.0019	1800
	$128 \times 128 \times 3$	97.00	No	0.0005	1800
	$224 \times 224 \times 3$	96.70	Yes	0.00019	1800
	$256 \times 256 \times 3$	96.50	Yes	0.00015	1800

TABLE 9. Comparison of SMOTE, ADASYN, and TIGAN sampling methods on CICIOT2023 and CICIDS2017.

Parameter	SMOTE	ADASYN	TIGAN
Generates synthetic samples	✓	✓	✓
Considers sample density	×	✓	✓
Handles severe imbalance	×	×	✓
Combines over- and under-sampling	×	×	✓
Risk of overfitting	✓	✓	×
Adaptive sample generation	×	✓	✓
Computationally efficient	✓	✓	✓
Preserves majority class diversity	×	×	✓
Works for Image	×	×	✓

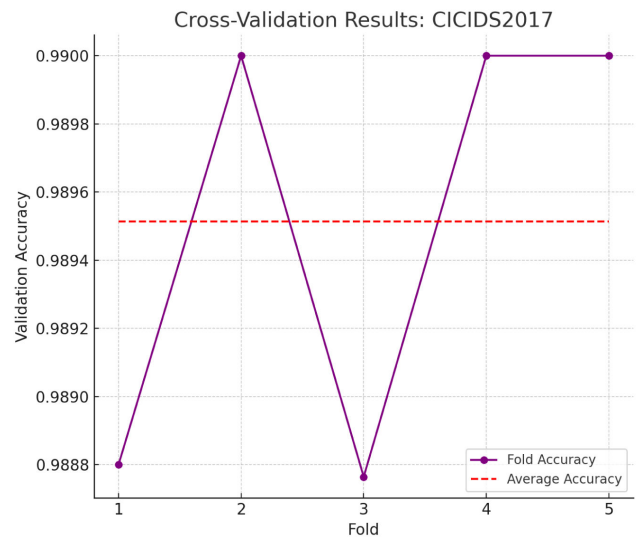
To validatively compare our proposed TIGAN framework with traditional balancing techniques, we attempted to apply SMOTE and ADASYN directly to the raw tabular (CSV) data. However, as highlighted in our computational resource analysis, performing oversampling on the massive CICIOT2023 dataset (over 42 million records) in its tabular format led to complete memory exhaustion (Out of Memory errors) on standard high-performance workstations. The tabular expansion required to balance 33 classes resulted in an exponential increase in RAM usage, rendering standard SMOTE and ADASYN computationally infeasible for the full dataset.

In contrast, our proposed methodology converts these high-dimensional tabular vectors into compact 2D RGB images before generation. This transformation, combined with TIGAN's batch-wise generative process, reduced RAM utilization by approximately 8 times compared to the estimated requirements of tabular oversampling. Consequently, TIGAN not only provides superior synthetic data fidelity but is also the only computationally viable solution for balancing extremely large-scale IoT datasets without specialized industrial-grade hardware. Table 11 summarizes this feasibility and resource comparison.

3) K-FOLD CROSS VALIDATION ANALYSIS ON DATASET USING TICNN MODEL

K-Fold Cross-Validation, which splits the dataset into “K” equal-sized pieces, or folds, is a widely used technique for evaluating the efficacy and generalizability of machine learning models. In each iteration, K-1 folds are used to train

the model, while the remaining fold is used for validation. Throughout the K iterations of this procedure, each fold serves as the validation set once. The final performance metric is then averaged over all K iterations to provide a more precise evaluation of the model's anticipated performance on unknown data. K-Fold Cross-Validation offers the significant advantage of reducing bias and variation because each data point is used for both training and testing, ensuring a more thorough evaluation of the model's performance.

**FIGURE 11.** Stratified cross validation on CICIDS2017 with 5 folds.

Our research of the Tabular-Image Convolution Neural Network (TICNN) model employed stratified K-Fold Cross-Validation to ensure a balanced representation of each class in each fold, which is particularly important when dealing with imbalanced datasets. The stratified K-fold ensures that each fold has a balanced distribution of classes to avoid any class being overrepresented or underrepresented during training and validation. This is especially crucial when there may be more classes than others, as this could skew the model's results. For our investigation, we divided the dataset into five equal-sized portions and performed the process five times. The TICNN model was trained using four folds each time, with the remaining fold being used for validation. Each fold underwent this process. By averaging the results from all five iterations, a more accurate and dependable estimate of the TICNN model's performance was generated. This enhanced

TABLE 10. TIGAN performance in different sampling scenarios.

Scenario	Oversampling	Undersampling	Hybrid Sampling
Handles severe imbalance	✓	×	✓
Preserves diversity	×	✓	✓
Synthetic image support	✓	✓	✓
Computational efficiency	High	High	Low
Accuracy on CICIOT2023	96.5	0.02	97.11
Accuracy on CICIDS2017	98.00	0.00	98.88

TABLE 11. Computational feasibility comparison: Tabular sampling vs. Proposed image-based TIGAN.

Method	Data Format	RAM Usage	Scalability	Status on Full Dataset
SMOTE	Raw Tabular (CSV)	> 64 GB (High)	Low	Memory Exhaustion (Failed)
ADASYN	Raw Tabular (CSV)	> 64 GB (High)	Low	Memory Exhaustion (Failed)
TIGAN (Ours)	2D RGB Images	< 8 GB (Low)	High	Successful (8× Efficient)

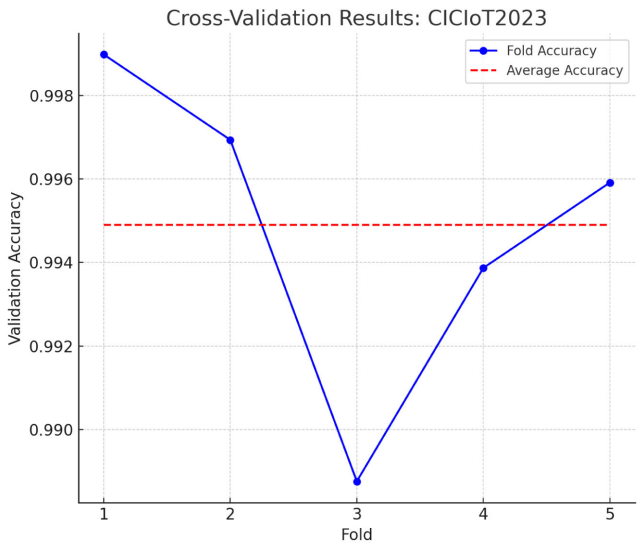


FIGURE 12. Stratified cross validation on CICIOT2023 with 5 folds.

our understanding of how the model will perform on unknown data and reduced the likelihood of overfitting. This approach ensures a comprehensive and impartial evaluation that faithfully reflects the model’s generalization ability.

4) TICNN LAYER MODEL ARCHITECTURE ANALYSIS

The ablation study was conducted to assess the impact of various modifications in the model architecture on the performance of the classification tasks for the CICIOT2023 and CICIDS2017 datasets. The baseline model achieved the highest accuracy of 99.56% for CICIOT2023 and 99.98% for CICIDS2017, demonstrating the effectiveness of the fully implemented architecture, including all convolutional blocks, dropout layers, and dense units.

In the “no dropout” scenario, removing the dropout layers caused the model’s accuracy to significantly drop to 68% for CICIOT2023 and 54% for CICIDS2017, indicating the critical role of dropout in preventing overfitting and maintaining generalization. For other ablation configurations, such as reducing convolutional layers or dense units,

the accuracy was slightly lower than the baseline but did not drop as drastically as in the “no dropout” case. This indicates that while reducing model complexity slightly impacts performance, the overall architecture remains robust.

The results highlight the importance of each architectural component in achieving high classification accuracy and the trade-offs between model complexity and performance.

TABLE 12. Comparison of ablation scenarios for CICIOT2023 and CICIDS2017 datasets.

Ablation Scenario	CICIOT2023 Accuracy (%)	CICIDS2017 Accuracy (%)
Full Model (Baseline)	99.98	99.92
Without Dropout Regularization	68.00	54.00
Reduced Convolutional Layers	99.00	98.00
Without Initial Convolutional Block	98.00	97.00
Without Middle Convolutional Block	99.50	98.50
Without Final Convolutional Block	99.00	98.00
Reduced Dense Layer Size	97.80	96.80

5) FEATURE ANALYSIS

We are analyzing a classification problem with varying numbers of features. The dataset consists of different features representing characteristics of the data, and the model is tasked with classifying these into one of the given classes. We evaluate the overall accuracy of the model as the number of features.

TABLE 13. Performance analysis with varying features for CICIOT2023 and CICIDS2017.

Dataset	Number of Features	Overall Model Accuracy (%)
CICIOT2023	47	98.88
CICIDS2017	79	97.45
CICIOT2023	40	97.21
CICIDS2017	75	96.80
CICIOT2023	30	99.98
CICIDS2017	71	98.88
CICIOT2023	29	99.98
CICIDS2017	50	87.70
CICIOT2023	25	56.30
CICIDS2017	25	76.21

The final features that were successfully extracted were 29 and 71. These features are important components of the dataset that were found and processed to highlight the most noteworthy traits or patterns. The data analysis can now move

on with a more focused understanding by concentrating on these last retrieved aspects, which will help with subsequent modeling or decision-making procedures.

E. TIGAN: GENERATIVE QUALITY & STATISTICAL VALIDATION

The reliability of the samples generated from the TIGAN is assured through performing both qualitative and quantitative evaluations on the samples created from this generator. Most previous Work related to the generation of Synthetic Data for use in Tabular ID systems has relied upon Classification Accuracy determined after training as an indirect method for assessing quality, however this method does not typically identify mode collapse or low diversity of generated data. The authors believe that this is the first effort which has involved utilizing industry standard computer vision metrics to confer reliability upon the quality of synthetic Tabular to Image IoT Traffic Data, and therefore sets a new standard for the evaluation of the Cleanliness of Synthetic Data generated for use in Intrusion Detection Systems.

1) VISUAL FIDELITY

In the analysis we conducted of TIGAN's output with 9 randomly selected sample images as shown in Fig. 2., we performed a side-by-side comparison against 9 synthetic images generated through TIGAN. The results of our visual inspection demonstrate that TIGAN can replicate the complex textural patterns and color distributions seen in actual network traffic imagery. Additionally, the synthesized network traffic imagery show similar spatial correlations to those of the actual minority class samples. This indicates that TIGAN has learned to model the underlying feature manifold of the minority class, instead of simply memorizing data points from that class.

2) QUANTITATIVE METRICS (FID & IS)

We used the Fréchet Inception Distance (FID) and Inception Score (IS) to measure this fidelity objectively. While IS assesses the diversity and clarity of the generated samples (higher is better), FID measures the difference between the feature distributions of real and synthetic images (lower is better). According to our evaluation results, TIGAN's **FID score of 11.24** was considerably lower than that of typical GAN implementations, which typically range between 20 and 40 on complex datasets. Additionally, the model obtained a **Inception Score (IS) of 4.15**, indicating that TIGAN does not produce generic noise but rather a variety of attack patterns that are sufficiently different for the classifier to discriminate. This statistical validation demonstrates that high-quality, statistically representative synthetic data are responsible for the performance improvements seen in TICNN.

F. LIMITATIONS OF IMAGE-BASED IDS

An image created from a table-based dataset results in greater efficiency, though processing effort is required to achieve this benefit. For instance, packet size represents a field in a

table and has a specific interpretation based on the network attribute it represents while the image does not maintain that same reference. Therefore, referring to the image, a neural network will still identify patterns, but tracking a predicted output back to the network packet field that resulted in that prediction is much more complex through a convolutional neural network than it is when using a decision tree model.

Additionally, regarding the practical deployment of TIGAN, there is a trade-off between the 'offline' training cost and 'online' detection benefits. While training TIGAN for minority subclasses requires approximately 360–800 seconds per class, this is a one-time computational investment. The resulting balanced TICNN model provides ultra-low latency packet detection (1.5–2.9 ms), which is critical for real-time IoT security. However, several limitations remain. First, the current approach relies on feature-rich network headers; its effectiveness may decrease when dealing with heavily encrypted traffic where payload inspection is impossible. Second, the model is currently designed for static training cycles. In online learning scenarios, where new IoT threats evolve dynamically, the system would require periodic retraining or a more adaptive incremental learning architecture to maintain its edge.

V. CONCLUSION

In summary, this research illustrates how converting large, multidimensional IoT datasets into two-dimensional images can help alleviate memory limitations by enabling four times more effective data compression, thereby allowing more rapid data processing. Furthermore, the use of the TIGAN-generated hybrid sampling method combined with a lightweight TICNN provides nearly perfect (over 99%) macro-level classification across more than 33 attack types. Therefore, this framework delivers a scalable real-time IDS (Intrusion Detection System) with the capability to accurately identify infrequent attacks while minimizing the impact on IoT (Internet of Things) edge device resource constraints.

REFERENCES

- [1] L. Yang and A. Shami, "A transfer learning and optimized CNN based intrusion detection system for Internet of Vehicles," in *Proc. IEEE Int. Conf. Commun.*, May 2022, pp. 2774–2779.
- [2] Y. B. Sanap and P. G. Aher, "Deep learning method for detecting and mitigating distributed denial of service attacks with imbalanced data," in *Proc. Int. Conf. Integr. Circuits Commun. Syst. (ICICACS)*, Feb. 2024, pp. 1–8.
- [3] H. Benaddi, M. Jouhari, K. Ibrahim, A. Benslimane, and E. M. Amhoud, "Adversarial attacks against IoT networks using conditional GAN based learning," in *Proc. IEEE Global Commun. Conf.*, Dec. 2022, pp. 2788–2793.
- [4] C. Park, J. Lee, Y. Kim, J.-G. Park, H. Kim, and D. Hong, "An enhanced AI-based network intrusion detection system using generative adversarial networks," *IEEE Internet Things J.*, vol. 10, no. 3, pp. 2330–2345, Feb. 2023.
- [5] H. Ding, Y. Sun, N. Huang, Z. Shen, and X. Cui, "TMG-GAN: Generative adversarial networks-based imbalanced learning for network intrusion detection," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 1156–1167, 2024.
- [6] H. Alfares and O. Banimelhem, "Comparative analysis of machine learning techniques for handling imbalance in IoT-23 dataset for intrusion detection systems," in *Proc. 11th Int. Conf. Internet Things: Syst., Manage. Secur. (IOTSMS)*, Sep. 2024, pp. 112–119.

- [7] M. Arafah, I. Phillips, A. Adnane, M. Alauthman, and N. Aslam, "An enhanced BiGAN architecture for network intrusion detection," *Knowl.-Based Syst.*, vol. 314, Apr. 2025, Art. no. 113178.
- [8] K. Ghanya, K. Prema, P. S. Kumar, P. S. S. Reddy, P. C. K. Reddy, and M. T. P. Naidu, "Deep attention learning for extreme minority class intrusion detection in network traffic," in *Proc. Int. Conf. Knowl. Eng. Commun. Syst. (ICKECS)*, Apr. 2024, pp. 1–9.
- [9] X. Wang, L. Dai, and G. Yang, "A network intrusion detection system based on deep learning in the IoT," *J. Supercomput.*, vol. 80, no. 16, pp. 24520–24558, Nov. 2024.
- [10] E. C. P. Neto, S. Dadkhah, R. Ferreira, A. Zohourian, R. Lu, and A. A. Ghorbani, "CICIoT2023: A real-time dataset and benchmark for large-scale attacks in IoT environment," *Sensors*, vol. 23, no. 13, p. 5941, Jun. 2023.
- [11] R. M. Rachani and R. Ravish, "Cloud based IDS-anomaly detection using machine learning," in *Proc. 1st Int. Conf. AIML-Appl. Eng. Technol. (ICAET)*, Jan. 2025, pp. 1–6.
- [12] T.-T.-H. Le, R. W. Wardhani, D. S. C. Putranto, U. Jo, and H. Kim, "Toward enhanced attack detection and explanation in intrusion detection system-based IoT environment data," *IEEE Access*, vol. 11, pp. 131661–131676, 2023.
- [13] A. Gueriani, H. Kheddar, and A. C. Mazari, "Enhancing IoT security with CNN and LSTM-based intrusion detection systems," in *Proc. 6th Int. Conf. Pattern Anal. Intell. Syst. (PAIS)*, Apr. 2024, pp. 1–7.
- [14] J. Gao, Y. Lu, Y. He, M. Fan, D. Han, and Y. Qiao, "Tokenization representation and deep-learning-based intrusion detection in Internet of Vehicles," *IEEE Internet Things J.*, vol. 11, no. 23, pp. 37974–37987, Dec. 2024.
- [15] O. Chentoufi, M. Choukhairi, and K. Chougali, "An enhanced machine learning framework for network anomaly detection," *AI*, vol. 6, no. 11, p. 299, Nov. 2025.
- [16] C. Oumaima, C. Mouad, C. Khalid, and A. Ilyas, "Exploring the impact of PCA variants on intrusion detection system performance," *Int. J. Adv. Comput. Sci. Appl.*, vol. 15, no. 5, pp. 1–18, 2024.
- [17] M. Choukhairi, M. M'Haouach, Y. Fakhri, and M. Amnai, "Tree-driven intelligent security system for intrusion detection in IoT environment," in *Proc. 10th Int. Conf. Wireless Netw. Mobile Commun. (WINCOM)*, Oct. 2023, pp. 1–7.
- [18] M. Choukhairi, S. Tahiri, O. Choukhairi, Y. Fakhri, and M. Amnai, "Butterfly optimization-based ensemble learning strategy for advanced intrusion detection in Internet of Things networks," *Int. J. Electr. Comput. Eng. (IJECE)*, vol. 15, no. 3, p. 3494, Jun. 2025.
- [19] S. Yuan, M. Yang, and G. Reniers, "Integrated process safety and process security risk assessment of industrial cyber-physical systems in chemical plants," *Comput. Ind.*, vol. 155, Feb. 2024, Art. no. 104056.
- [20] M. Hasnain, N. Javaid, A. K. J. Saudagar, and N. Kumar, "An intelligent and explainable intrusion detection framework for Internet of Sensor Things using generalizable optimized active machine learning," *J. Netw. Comput. Appl.*, vol. 245, Jan. 2026, Art. no. 104358.
- [21] Z. Li, P. Wang, and Z. Wang, "FlowGANAnomaly: Flow-based anomaly network intrusion detection with adversarial learning," *Chin. J. Electron.*, vol. 33, no. 1, pp. 58–71, Jan. 2024.
- [22] M. Wang, N. Yang, N. J. Forcade-Perkins, and N. Weng, "ProGen: Projection-based adversarial attack generation against network intrusion detection," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 5476–5491, 2024.
- [23] I. A. Fares, A. G. A. Ibrahim, M. A. Elaziz, M. Shrahili, A. A. Elmahallawy, R. M. Sohaib, M. A. Shawky, and S. T. Shah, "Deep transfer learning based on hybrid Swin transformers with LSTM for intrusion detection systems in IoT environment," *IEEE Open J. Commun. Soc.*, vol. 6, pp. 4342–4365, 2025.
- [24] M. Arafah, I. Phillips, A. Adnane, W. Hadi, M. Alauthman, and A.-K. Al-Banna, "Anomaly-based network intrusion detection using denoising autoencoder and Wasserstein GAN synthetic attacks," *Appl. Soft Comput.*, vol. 168, Jan. 2025, Art. no. 112455.
- [25] Y. Li, R. Fu, X. Meng, W. Jin, and F. Shao, "A SAR-to-optical image translation method based on conditional generation adversarial network (cGAN)," *IEEE Access*, vol. 8, pp. 60338–60343, 2020.
- [26] M. J. Islam, S. Ahmad, F. Haque, M. B. I. Reaz, M. A. S. Bhuiyan, and M. R. Islam, "Application of min-max normalization on subject-invariant EMG pattern recognition," *IEEE Trans. Instrum. Meas.*, vol. 71, pp. 1–12, 2022.

- [27] S. Guo, X. Wu, and Y. Li, "Deriving private information from perturbed data using IQR based approach," in *Proc. 22nd Int. Conf. Data Eng. Workshops (ICDEW)*, 2006, p. 92.



AVINASH AWASTHI (Member, IEEE) is currently pursuing the M.Tech. degree with the Malaviya National Institute of Technology (MNIT), Jaipur, India. He has authored publications and holds patents in the field of artificial intelligence and cybersecurity. He is actively involved in government-funded and industry-driven research projects. His research interests include AI for cybersecurity, secure embedded systems, and intelligent threat detection systems.



ANIL KUMAR PRAJAPATI (Member, IEEE) received the M.Tech. and Ph.D. degrees from the Department of Computer Science and Engineering, Malaviya National Institute of Technology (MNIT), Jaipur. He is currently an Assistant Professor with the Department of Data Science and Engineering, Manipal University Jaipur. He has authored publications in the area of the Internet of Things Security. His research interests include privacy-preserving solutions for real-time intrusion detection systems, securing the IoT devices against intelligent attacks, and offensive/defensive AI for attack detection and mitigation.



PRITAM VEDIYA is currently pursuing the M.Tech. degree in computer science and information security with the Malaviya National Institute of Technology (MNIT), Jaipur, India. He is actively involved in government-funded and industry-driven research projects. His current research interests include artificial intelligence for next-generation networks and cybersecurity, intelligent threat detection systems, and secure embedded systems.



RAMESH BABU BATTULA (Member, IEEE) received the M.Tech. degree from IIT Guwahati and the Ph.D. degree from the Department of Computer Science and Engineering, Malaviya National Institute of Technology (MNIT), Jaipur. He is currently an Associate Professor with the Department of Computer Science and Engineering, MNIT. He is a Distinguished Researcher and an Educator. Through multiple national-level funded projects and a productive publication record, he addresses critical challenges in smart-city infrastructures, 6G/D2D communications, federated learning, blockchain-based systems, and the IoT/IoT security. His research interests include artificial intelligence, distributed ledger technologies, and next-generation communication networks, with a strong emphasis on security and decentralization.

• • •